

Interactive Target Element Selection in Infographics via Scene Tree Modeling

Jiangning Zhu, Yuxing Zhou, Lei Li, Shixia Liu

Abstract—Accurately detecting infographic elements is essential for infographic understanding and generation. However, high-performance detection models rely on large-scale training datasets with precise bounding-box annotations for infographic elements, which are time-consuming to create manually. Although fine-tuned object detection models can predict candidate bounding boxes, they often include non-target elements to be filtered out and therefore require efficient target element selection. To address this challenge, we develop a human-in-the-loop tool, EleDetective, that combines automatic selection, coordinated visualizations, and interactive refinement. The key idea is to represent an infographic as a scene tree that captures the region hierarchy. Based on this representation, we formulate target element selection as a bilevel optimization problem to recover missed elements and remove misselected elements. Since automatic selection is not always perfect, we introduce a treemap-based grid visualization and a scene tree visualization to help users explore selection results and identify potential errors. Users then correct the identified errors, and their corrections are propagated across infographics based on query-sensitive similarity, improving the quality of selection at scale. Quantitative evaluation and two use cases demonstrate the effectiveness of our method.

 **Code:** <https://github.com/EleDetective/EleDetective>

Index Terms—Infographics, object detection, element selection, scene tree, grid visualization.

I. INTRODUCTION

Infographics combine visual elements, such as icons, pictograms, and charts, with textual elements to present complex information in an engaging and accessible way. As a result, they are widely used for data communication in various domains, including journalism, marketing, and education. Accurate infographic element detection is essential for downstream tasks such as understanding and generation. For infographic understanding, more complete and precise detection of bars and axes helps vision-language models (VLMs) associate each bar with the corresponding axis and infer their values more accurately [1], [2]. For infographic generation, accurate element detection helps identify recurring layout patterns and derive reusable layout templates for constructing new ones [3], [4]. However, developing high-performance detection models requires large-scale training datasets with precise bounding-box annotations for infographic elements. Because manual annotation requires domain expertise and is time-consuming, a practical alternative is to use a fine-tuned object detection

model to predict candidate bounding boxes for infographic elements. Prior work on object detection has shown that such model-assisted verification can substantially reduce annotation effort (e.g., from 53 hours to 5.8 hours for about 4,500 elements [5]). However, the predicted candidate bounding boxes often include both true target elements to be retained as valid annotations and non-target elements to be filtered out. Thus, target element selection is needed before these candidates can be used for model training. As shown in Fig. 1(a), a common strategy is to select candidates with confidence scores above a predefined threshold. However, confidence scores alone cannot reliably distinguish target elements from non-target ones [6]. Even with a carefully chosen threshold, this strategy can still miss target elements and incorrectly retain non-target candidates (Fig. 1A). This makes it challenging to determine which candidates should be retained as final annotations.

To address this challenge, we propose a human-in-the-loop tool, EleDetective, that helps dataset curators prepare training datasets for infographic element detection by identifying target elements from candidate bounding boxes (Fig. 1(b)). The tool centers on a scene tree representation [7], which captures the region hierarchy of each infographic. By organizing candidate bounding boxes into a hierarchy, this representation supports region-level evaluation of coverage and overlap. Based on this representation, we formulate target element selection as a bilevel optimization problem to recover missed elements and remove misselected elements. Since automatic selection is not always perfect, we design two visualizations to help users explore the selection results: a treemap-based grid visualization and a scene tree visualization. In the treemap-based grid visualization, we develop a layout method that provides an overview by clustering infographics with similar selection results. The key feature of this method is its ability to enhance cluster perception by mapping each cluster to a rectangular area, while jointly optimizing compactness and proximity. In the scene tree visualization, we employ a node-link diagram to show how candidate bounding boxes are grouped, helping users examine and compare candidates corresponding to the same target element more efficiently. The two visualizations work together to support interactive identification of selection errors. The treemap-based grid visualization highlights clusters likely to contain errors, including missing, misselected, and misclassified elements. Users can explore infographics within these clusters to select those with potential errors. Once an infographic is selected, the scene tree visualization enables users to locate error-related elements. Users then correct the corresponding selection errors, and their corrections are propagated across infographics based on

This work was supported by the National Natural Science Foundation of China under grants U21A20469. (Jiangning Zhu and Yuxing Zhou are co-first authors.) (Corresponding author: Shixia Liu.)

Jiangning Zhu, Yuxing Zhou, Lei Li, and Shixia Liu are with the School of Software, BNRist, Tsinghua University, Beijing 100084, China (e-mail: {zjn23, zhoyx23, lilei23}@mails.tsinghua.edu.cn, shixia@tsinghua.edu.cn).

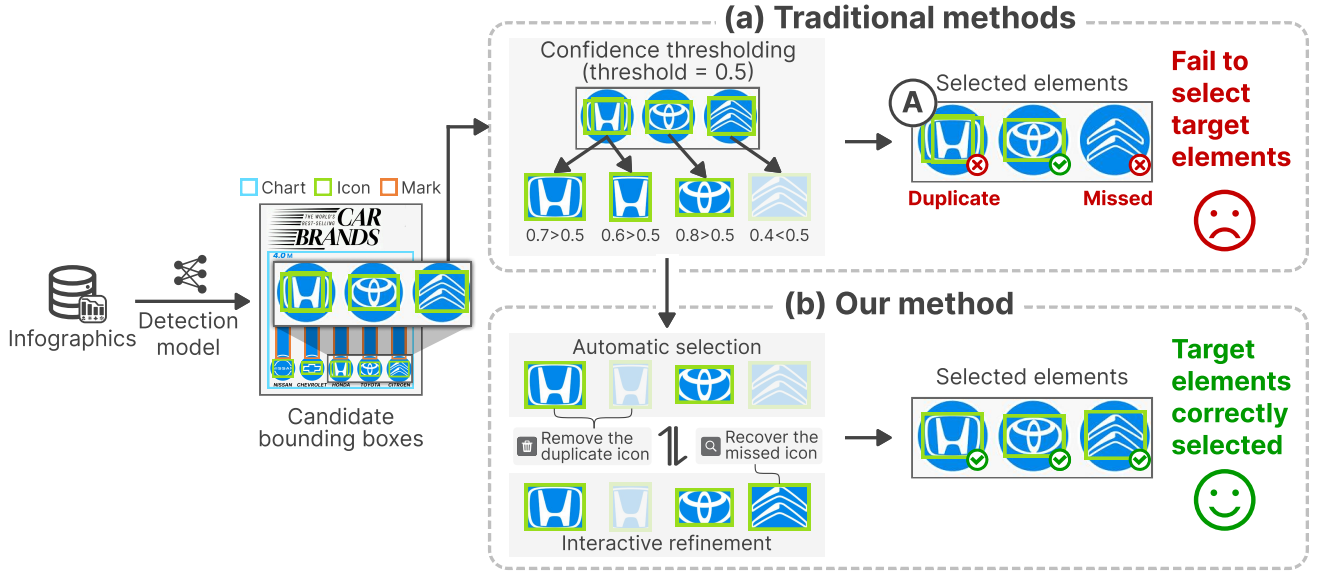


Fig. 1. The comparison between target element selection using traditional methods and the proposed human-in-the-loop method: (a) the traditional methods select candidate bounding boxes with confidence scores above a predefined threshold and struggle to identify the target elements; (b) our method leverages both automatic selection and interactive refinement to effectively identify the target elements.

query-sensitive similarity [8], leading to improved selection results for another round of analysis. In this way, EleDetective integrates automatic selection, coordinated visualizations, and interactive refinement into an iterative workflow for refining target element selection results.

We conducted quantitative experiments to show the effectiveness of the scene-tree-based automatic selection and the treemap-based grid layout. We also conducted two use cases to demonstrate EleDetective’s usefulness. In the first use case, EleDetective helped refine training data for infographic element detection. The post analysis showed that EleDetective supported more effective annotation refinement, and that the refined model transferred better to out-of-domain datasets. In the second use case, EleDetective helped improve visually grounded reasoning for infographic understanding.

The main contributions of this work include:

- A scene-tree-based method for automatically selecting target elements from candidate bounding boxes.
- A coordinated visualization design that combines a treemap-based grid layout and a scene tree visualization to support cluster-level identification and instance-level examination of selection errors.
- A human-in-the-loop tool, EleDetective, that integrates automatic selection, coordinated visualizations, and interactive refinement to support the refinement of target element selection results.

II. RELATED WORK

A. Sample Visualization

Sample visualizations display a collection of individual data samples in a 2D layout to support sample inspection and comparison [9]–[11]. They are widely used in data analysis, commonly taking the form of grid layouts [9], [12]–[16], treemaps [17]–[20], circle packing [21], and Voronoi diagrams [22]. Our goal is to display uncropped infographics and

reveal cluster structure. Grid layouts and treemaps best support these goals, so we review related work in these categories.

Existing grid layout methods can be classified into two categories [23]: direct mapping methods [12]–[14] and projection-based methods [9], [15], [16]. Direct mapping methods assign high-dimensional samples directly to grid cells. For example, Kernelized Sorting [12] formulates the assignment as a quadratic assignment problem that maximizes the correlation between pairwise data distances and grid distances. In contrast, projection-based methods first project the samples onto a 2D space and then assign them to grid cells. Isomatch [15] uses Isomap [24] for projection and formulates the assignment as weighted bipartite graph matching. Recent work has focused on enhancing cluster perception in grid layouts. Cluster-aware grid layout [25] employs global optimization to balance proximity and compactness, then refines convexity through boundary cell swapping. To support fuzzy cluster analysis, Zhou *et al.* [26] extended this method by placing ambiguous samples near cluster boundaries and preserving stability during exploration. Despite these efforts, clusters often remain non-rectangular, leaving room for improvement.

Treemap layout methods offer an alternative by partitioning the 2D space into rectangular areas, with compactness as a primary goal. Spatially Ordered Treemaps (SOT) [17] uses the squarified treemap layout [27] to generate near-square areas while preserving proximity. To further enhance compactness, Weighted Maps [18] and Nmap [19] adopt a recursive partitioning strategy. Recently, Dendromap [20] utilizes treemaps for hierarchical cluster visualization by building a binary tree and applying an improved slice-and-dice layout. However, it does not explicitly optimize the proximity of samples within clusters, which may limit the examination of consistent patterns among similar samples within a cluster.

Grid layouts are effective in preserving proximity, while treemap layouts are well-suited to creating convex areas. In

this work, we combine the advantages of both and propose a treemap-based grid layout method. The key feature of this method is that it maps clusters to compact, convex rectangular areas to enhance cluster perception, while preserving both the relative positions of clusters and the proximity of samples within each cluster.

B. Human-in-the-Loop Annotation Refinement

Human-in-the-loop methods have been widely used to refine model-predicted annotations. According to the annotation granularity, they can be grouped into two categories: sample-level label analysis and region-level annotation analysis.

Sample-level label analysis methods aim to improve labels associated with individual samples. Some methods directly support user correction of sample-level labels. For example, Snyder *et al.* [28] introduced a learning framework in which a classifier is updated in real time as users correct relevance labels assigned to tweets. V-awake [29] supports correcting predicted sleep-stage labels by displaying them along the time axis together with confidence scores and neighboring stages. VASSL [30] supports scalable manual labeling of spambots by comparing groups of accounts through multiple coordinated views. Other methods improve sample-level labels indirectly by refining the label assignment process. For example, DataLinker [22] supports interactive graph construction to improve label propagation in graph-based semi-supervised learning. FSLDiagnator [31] helps diagnose ensemble few-shot classifiers by identifying ineffective base learners and low-quality labeled examples. While these methods are effective for improving sample-level labels, they are less suitable for region-level annotations, which pair labels with regions rather than the entire sample.

Region-level annotation analysis methods consider annotations that pair labels with regions, such as bounding boxes and masks. Some focus on diagnosing the behavior of models that predict region-level annotations. For detection, VATLD [32] combines disentangled representation learning and semantic adversarial learning to reveal issues in accuracy and robustness. To interpret detection failures from the perspectives of what, why, and how, Wang *et al.* [33] combined the analysis of macro-level spatial-temporal context and micro-level model features. Uni-Evaluator [34] models discrete class labels and continuous bounding boxes as probability distributions to support a unified diagnosis of diverse tasks. For segmentation, VASS [35] diagnoses model failures based on spatial properties and scene contexts. VISTA [36] helps users inspect and correct irrelevant labels and label-region misalignments. Other methods improve the models that predict region-level annotations by incorporating external knowledge. For example, MutualDetector [37] integrates caption supervision with object detection, improving both through mutual refinement. OW-Adapter [38] supports open-world detection by incorporating user feedback to identify novel object classes. The above methods generally organize region-level annotations as flat collections. In contrast, EleDetective organizes candidate elements within each infographic into a scene tree. This hierarchical representation enables region-level evaluation of coverage

and overlap for automatic selection, supports cluster-level error analysis through scene-tree embeddings and instance-level examination through element hierarchies, and provides contextual information for correction propagation.

III. DESIGN OF ELEDetective

A. Requirement Analysis

We collaborated with three machine learning experts (E1, E2, E3) to design EleDetective. E1 is a senior researcher at a major IT company with a research interest in infographic generation. E2 and E3 are two Ph.D. students working on infographic understanding and object detection. None of them is a co-author of this paper. From three 60-minute semi-structured interviews (one per expert) and a literature review [1], [4], [6], [22], [34], we derived the following requirements.

R1. Provide an overview to identify selection errors at scale. In practice, dataset curators often prepare training datasets for detection tasks by using a fine-tuned object detection model to predict candidate bounding boxes and retaining candidates with confidence scores above a fixed threshold [1], [4]. However, this strategy often misses target elements and incorrectly retains non-target candidates. Therefore, dataset curators need to manually examine the selection results. When working with large-scale datasets, such manual examination becomes infeasible. To address this issue, the experts emphasized the need for an effective overview of the selection results that facilitates error identification at scale. E1 commented, “It would be helpful to cluster similar selection results and show representative examples within each cluster. This makes it easier to identify clusters likely containing errors.”

R2. Facilitate efficient correction of identified errors. Once errors are identified, the experts proceed to correct them. However, this correction process is often time-consuming. As E3 explained, “Object detection models often predict redundant candidate bounding boxes that are highly overlapping, making it difficult to compare them and identify the target ones.” To address this challenge, candidate bounding boxes can be organized into a hierarchy that groups those corresponding to the same target element, enabling experts to examine and compare them more efficiently. Additionally, E3 suggested displaying key information about the candidates, such as their confidence scores, to aid in selecting the most appropriate one.

R3. Enable reliable propagation of corrections. All three experts emphasized the importance of propagating their corrections to reduce repeated work. This is particularly important because infographics often contain repeated or visually similar

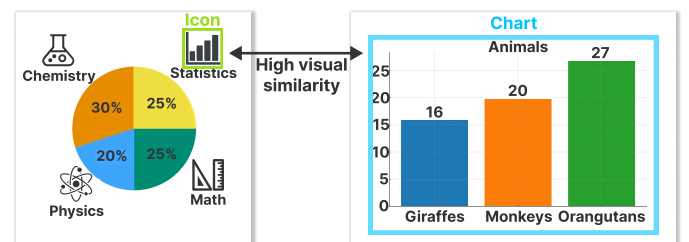


Fig. 2. Relying solely on visual similarity leads to incorrect propagation between elements from different categories.

elements, such as national flag icons, where propagation can substantially reduce correction efforts. However, E2 reported a difficulty in the propagation: “Deciding where to propagate a correction is difficult, as the visual complexity of infographics makes it hard to assess element similarity.” For example, as shown in Fig. 2, a chart icon is visually similar to an actual chart. However, corrections should not be propagated between their bounding boxes, as they belong to different categories. Therefore, propagation decisions should consider not only the visual similarity between elements but also their surrounding elements, which provide contextual information to distinguish visually similar but semantically different elements.

B. System Overview

Motivated by the requirements, we have developed EleDetective, a human-in-the-loop tool for target element selection in infographics. As shown in Fig. 3, it consists of three modules: automatic selection, coordinated visualizations, and interactive refinement. Starting from candidate bounding boxes predicted by the object detection model, the automatic selection module selects target elements via bilevel optimization. The selection results are then presented in two coordinated visualizations: a treemap-based grid visualization that summarizes selection results to help users identify infographics with potential errors (**R1**), and a scene tree visualization that supports detailed examination of elements in an identified infographic (**R2**). The interactive refinement module enables users to correct selection errors and reliably propagates these corrections across elements using query-sensitive similarity, which integrates both visual and contextual features (**R3**). These corrections are applied to iteratively refine the selection results.

IV. SCENE-TREE-BASED AUTOMATIC SELECTION

In this section, we first introduce the scene tree construction for infographics. We then formulate target element selection as a bilevel optimization problem based on the scene tree representation and solve it using a hill-climbing algorithm.

A. Scene Tree Construction

As shown in Fig. 4(a), candidate elements detected by the model correspond to leaf nodes in the scene tree. Following common practice in infographic analysis [4], [39], we define leaf elements as object-level elements, such as charts, axes, legends, textual elements, icons, and pictorial images. Compared with low-level graphical primitives such as lines and points, these elements serve as more appropriate semantic-level units for infographic understanding and generation [2], [40]. The elements are progressively grouped into the regions in the infographic, which correspond to the non-leaf nodes. We group elements following Koch *et al.*'s method [41], which applies three Gestalt-principle-inspired rules (Fig. 4(b)): 1) proximity, grouping spatially close elements or regions; 2) consistent whitespace, grouping those separated by equal spacing; and 3) alignment, grouping those with aligned boundaries. At each non-leaf node, these rules are applied to compute a local cost for grouping the elements or regions. These local costs are summed across the scene tree to produce a global cost $g(\mathcal{T}, E)$, which quantifies the inconsistency between the scene tree $\mathcal{T}$ and the hierarchical relationships among the elements in E . To minimize the global cost, we employ a greedy strategy that iteratively groups elements or regions with local costs below a threshold, progressively increasing the threshold to form the hierarchy.

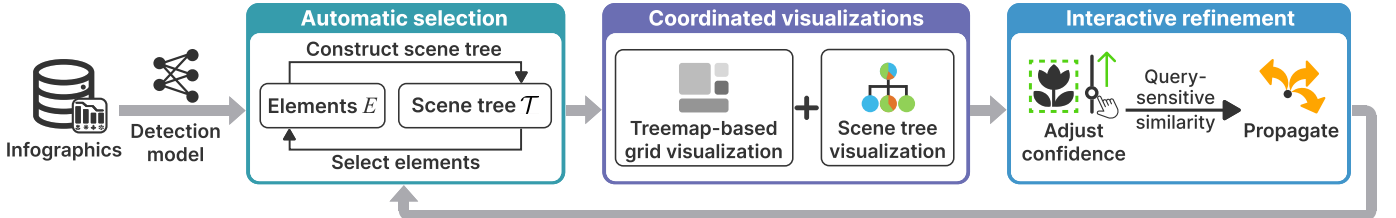


Fig. 3. EleDetective overview. Starting from candidate bounding boxes predicted by the object detection model, the automatic selection module identifies a set of target elements via bilevel optimization. The coordinated visualization design that combines a treemap-based grid layout and a scene tree visualization then supports identifying errors in the automatic selection. The interactive refinement module enables users to correct identified errors and propagate the corrections based on query-sensitive similarity.

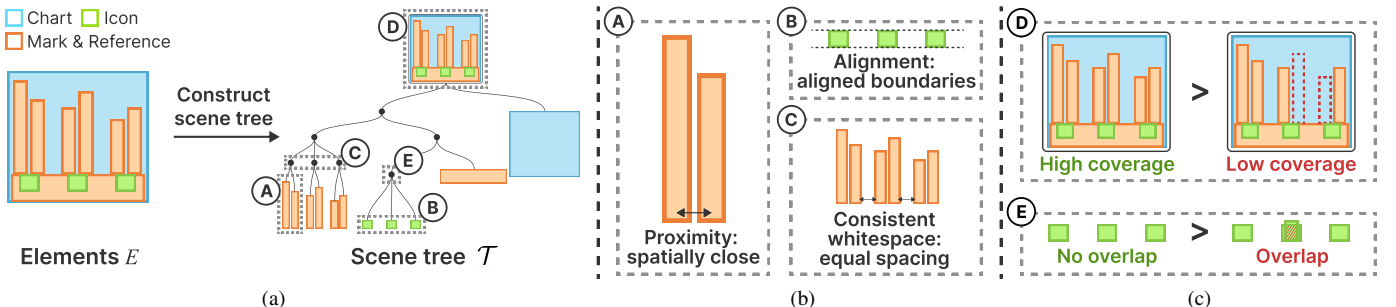


Fig. 4. Scene tree construction and target element selection: (a) the scene tree is constructed by progressively grouping the elements; (b) the Gestalt-principle-inspired rules for scene tree construction; (c) the maximal coverage and minimal overlap criteria for target element selection.

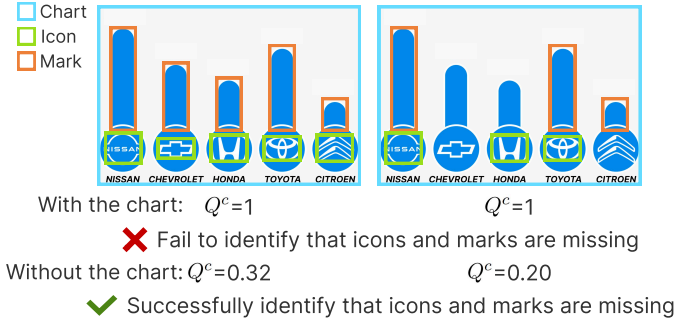


Fig. 5. Coverage computation with and without the large chart region.

B. Target Element Selection via Bilevel Optimization

A scene tree constructed from target elements satisfies two criteria: maximal coverage and minimal overlap (Fig. 4(c)).

The **maximal coverage criterion** requires that each region is as fully covered as possible by its child nodes' regions. Because regions in an infographic are typically filled with meaningful content, low coverage often indicates missing elements. A natural measure of coverage is the ratio between the union of the child nodes' regions and the parent's region. However, this can be misleading when one child node's region contains all its siblings' regions. For example, as shown in Fig. 5, in a chart with several smaller icons and marks, the larger chart region can mask the contribution of the icons and marks (its siblings), making it difficult to identify if any icon or mark is missing. To avoid this, we exclude such regions and define the coverage of a node v as $Q^c(v) = |\bigcup_{v' \in \mathcal{D}(v)} r(v')| / |r(v)|$, where $\mathcal{D}(v)$ is the set of child nodes of v excluding those whose regions contain all other siblings' regions, $r(v)$ is the region of the node v , and $|r(v)|$ is its size.

The **minimal overlap criterion** requires that there is no large overlap between the non-nested regions of sibling nodes. Since such regions seldom overlap substantially in infographics, violation of this criterion often indicates misselected elements. To measure the degree of overlap, we exclude child nodes whose region fully contains all its siblings' regions and calculate the overlap among the other regions as $Q^o(v) = \sum_{\{v'_1, v'_2\} \subset \mathcal{D}(v)} \mathbb{I}[|r(v'_1) \cap r(v'_2)| / |r(v'_1) \cup r(v'_2)| > 0.5]$, where $\mathbb{I}$ is an indicator function that returns 1 if the intersection-over-union (IoU) between two different regions $r(v'_1)$ and $r(v'_2)$ exceeds 0.5, and 0 otherwise. We choose 0.5 as the threshold following the common practice for identifying duplicates [6].

Combining the above criteria, we formulate the target element selection as a bilevel optimization problem. The upper level selects a subset of candidates $E' \subset E$ to maximize confidence and coverage while minimizing overlap, and the lower level constructs the scene tree $\mathcal{T}$ that best captures their hierarchical structure:

$$\begin{aligned} \max_{E' \subset E} \quad & \sum_{e' \in E'} (s(e') - \tau) + \lambda_1 \sum_{v \in \mathcal{T}} Q^c(v) - \lambda_2 \sum_{v \in \mathcal{T}} Q^o(v) \\ \text{s.t.} \quad & \mathcal{T} = \underset{\mathcal{T}}{\operatorname{argmin}} g(\mathcal{T}, E'). \end{aligned} \quad (1)$$

Here, $s(e')$ is the confidence score of an element e' . The threshold τ encourages selecting elements with scores above it and penalizes selecting those with scores below it. λ_1 and

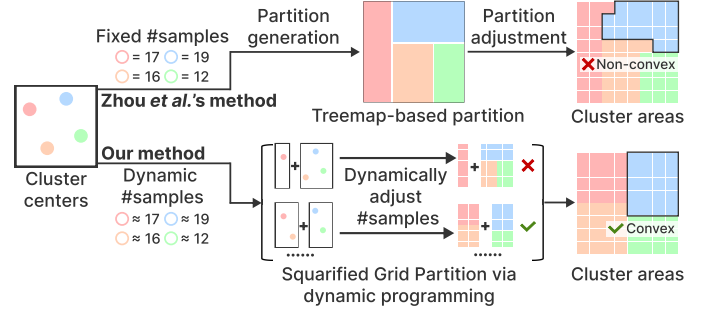


Fig. 6. Comparison between Zhou et al.'s method [26] targeting convex areas and our treemap-based grid layout method.

λ_2 are the weights to balance the three terms, which are determined by grid search. We solve this bilevel optimization problem using a hill-climbing algorithm that iteratively updates the selected candidates and constructs the corresponding scene tree to evaluate the objective.

V. ELEDETECTIVE VISUALIZATION

To support the exploration of the element selection results, we develop two coordinated visualizations: 1) a treemap-based grid visualization that provides an overview of the selection results across infographics (**R1**); and 2) a scene tree visualization that supports detailed examination of individual element selections within an infographic (**R2**). We then provide a set of rich interactions to support interactive refinement.

A. Treemap-Based Grid Visualization

We cluster infographics based on their scene-tree embeddings extracted using a Tree-LSTM model [42]. For a clearer view of these clusters, we develop a treemap-based grid visualization method that maps each cluster to a rectangular area and displays infographics sampled from the cluster in this area. In this visualization, cell colors encode the average confidence score of the elements within the corresponding infographic, with darker colors indicating lower scores that suggest possible errors (Fig. 9(a)). We also display representative selection results within each cluster and mark their positions in the grid with map pins (📍). The layout of clusters and samples is central to this visualization, which is detailed below.

As shown in Fig. 6, existing grid layout methods targeting convex cluster areas are limited by fixing the number of displayed samples [25], [26]. This fixed number often cannot be expressed as a product of row and column numbers, causing clusters to remain non-rectangular. To address this issue, we dynamically adjust the number of infographics sampled for presenting the overview of each cluster and generate the layout through a two-step optimization strategy comprising **treemap creation** and **sample assignment**. The treemap creation step partitions the grid into rectangular areas corresponding to each cluster, while optimizing their compactness and proximity. Then, the sample assignment step assigns samples to grid cells within each area while preserving their proximity.

Treemap creation. This step includes two parts: 1) cluster localization, which maps the cluster centers onto the grid while

preserving their proximity; and 2) grid partition, which creates compact rectangular areas around the cluster centers.

Cluster localization. We preserve cluster proximity by aligning the distances between their centers with their ideal distances:

$$\min_{\mathbf{x}, \mathbf{y}} \sum_{i \neq j} \frac{1}{d_{ij}^2} (\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} - d_{ij})^2, \quad (2)$$

where (x_i, y_i) is the i -th cluster center and d_{ij} is the ideal distance between the i -th and j -th cluster centers, calculated as the distance between their average scene-tree embeddings. Following the Kamada-Kawai layout method [43], the weights $\frac{1}{d_{ij}^2}$ emphasize preserving proximity between nearby clusters. The problem is solved using the force-directed method [44].

Grid partition. Inspired by treemap visualization [45], we iteratively split the clusters into two subsets and, correspondingly, split the grid area into two subareas. At each step, one of the following operations is applied: 1) the clusters are split by the y -coordinates of their centers, and the area is split by a horizontal cut; or 2) the clusters are split by the x -coordinates of their centers, and the area is split by a vertical cut. Since the area size should be proportional to the number of samples, the cluster split uniquely determines the area split. Specifically, for an area of height h and width w containing m clusters, each with a_i samples, the possible splits are:

- cluster subsets $S_1^y = \{i | y_i \leq y_{\text{split}}\}$ and $S_2^y = \{i | y_i > y_{\text{split}}\}$, resulting in two subareas with width w and heights $h_1 = \text{round}(h \cdot \sum_{i \in S_1^y} a_i / \sum_{i=1}^m a_i)$, $h_2 = h - h_1$; and
- cluster subsets $S_1^x = \{i | x_i \leq x_{\text{split}}\}$ and $S_2^x = \{i | x_i > x_{\text{split}}\}$, resulting in two subareas with height h and widths $w_1 = \text{round}(w \cdot \sum_{i \in S_1^x} a_i / \sum_{i=1}^m a_i)$, $w_2 = w - w_1$.

Here, $\text{round}(w)$ rounds w to the closest integer, slightly adjusting the number of displayed samples so that cells are not split across subareas. To optimize cluster compactness, we evaluate all possible horizontal and vertical splits and choose the one that produces the most compact clusters:

$$F(h, w, S) = \begin{cases} \min \left(\min_{y_{\text{split}}} (F(h_1, w, S_1^y) + F(h_2, w, S_2^y)), \right. \\ \quad \left. \min_{x_{\text{split}}} (F(h, w_1, S_1^x) + F(h, w_2, S_2^x)) \right), & |S| > 1, \\ \sum_{i=1}^h \sum_{j=1}^w \|(i, j) - ((h+1)/2, (w+1)/2)\|^2, & |S| = 1, \end{cases} \quad (3)$$

where $F(h, w, S)$ denotes the optimal compactness when partitioning an area of height h and width w for clusters in S . For $|S| = 1$, compactness is measured as the total distance from the area center to its cells, following Rottmann *et al.* [13]. To solve Eq. (3), we employ dynamic programming with memoization that recursively explores all possible splits while caching results to avoid redundant exploration.

Sample assignment. For an area with n cells, we randomly select n samples for display. Following Zhou *et al.* [26], we preserve the proximity between samples by minimizing the KL divergence between the similarity distributions in the high-dimensional space and those on the 2D grid. We solve this minimization using a standard solver FAQ [46].

B. Scene Tree Visualization

To facilitate detailed examination of the selection results within a single infographic, we visualize its scene tree as a

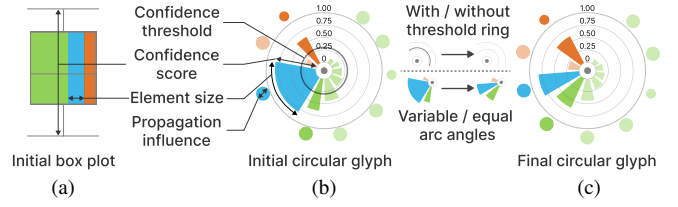


Fig. 7. The design of nodes in the scene tree visualization: (a)-(b) alternative designs; (c) final design.

node-link diagram (Fig. 9(b)). Each node is positioned along the y -axis according to the average confidence of the elements in its region, thereby highlighting low-confidence regions for further inspection. The node also summarizes key statistics of its region, which is detailed below.

Given the importance of the confidence score in the selection process, our initial design utilizes a box plot to present the distribution of confidence scores at each node (Fig. 7(a)). The box width encodes the total element size, and the box is further divided into segments representing the proportion of each category. The experts agreed that this design facilitates identifying elements with low confidence scores. However, to further reduce manual effort, they also sought to identify elements whose correction could substantially affect other elements during propagation.

To meet this need, we replaced the box plot with a circular glyph (Fig. 7(b)). It comprises: 1) a circular bar chart, where each bar represents an element, with bar length encoding confidence score and arc angle encoding element size; and 2) a surrounding ring of markers, where the radius of each marker encodes propagation influence (*i.e.*, the total similarity of the element with other elements). To assist confidence score comparison, we include concentric reference rings as visual guides and add a highlighted one that corresponds to the confidence threshold for the element selection. Bars and markers are color-coded by category, with selected elements shown at full opacity and unselected elements at reduced opacity. Expert feedback suggested that this design was visually complex and less interpretable at a glance. Accordingly, we simplified the design by using equal arc angles for all bars, as element size was less relevant to correction, and removing the threshold ring, which served mainly as an auxiliary reference. This resulted in the final design (Fig. 7(c)).

C. Interactive Refinement

To support the refinement of selection results, we provide a set of rich interactions.

Hierarchical grid exploration. When handling large-scale datasets, it is necessary to progressively select and visualize subsets of the infographics within the grid layout. To achieve this, we adopt the hierarchical exploration method of Zhou *et al.* [26], which organizes all infographics into a hierarchy. Initially, only top-level clusters with their representative infographics are displayed in the grid. Users can zoom into a selected rectangular region, and the grid is updated to display the infographics in that region together with a randomly sampled subset of their descendants. In this way, all infographics

can be inspected through successive zoom-in operations. To preserve the mental map, the relative positions of the clusters are maintained throughout the exploration. If an infographic warrants further examination, users can click on it to examine its details in the scene tree visualization.

Scene tree exploration. Since infographics often contain many elements, the corresponding scene trees can include a large number of leaf and non-leaf nodes, making it difficult to display all nodes simultaneously without causing visual clutter. To address this, we adopt a focus-plus-context strategy, displaying only nodes within a predefined distance of the focused node, while collapsing the rest to maintain clarity. For example, as shown in Fig. 9(b), when the user focuses on node C1, the nodes around C1 are displayed to reveal the scene tree structure, whereas nodes under C2 are collapsed to avoid clutter. Users begin exploration from the root of the scene tree and navigate by clicking nodes until they locate selections that require correction.

Selection correction and propagation. Users may identify three types of errors: missed, misselected, and misclassified elements. Clicking an element opens a floating panel (Fig. 9C3) for correction: 1) a missed element can be added by clicking $\oplus$; 2) a misselected element can be removed by clicking $\ominus$; and 3) a misclassified element can be corrected by selecting the correct category from a dropdown list, which contains the categories supported by the detection model.

To reduce correction effort, user corrections are propagated among relevant infographics (**R3**). The key observation enabling the propagation is that a correction can be expressed as an update to the confidence score: increasing it for a missed element, decreasing it for a misselected element, and setting it to 0 while creating a new element under the correct category for a misclassified element. Accordingly, we propagate a correction by adjusting the confidence scores of similar elements, with the adjustment strength determined by their similarity to the corrected element:

$$\Delta s(e_j) = \Delta s(e_i) \cdot \text{Sim}(e_i, e_j | z_i), \quad (4)$$

where $\Delta s(e_i)$ is the change to element e_i due to user correction and $\Delta s(e_j)$ is the propagated change in confidence score. $\text{Sim}(e_i, e_j | z_i)$ is the query-sensitive similarity [8] between e_i and e_j , which combines visual and contextual similarities by treating z_i , the contextual information of e_i , as a query. It is defined as:

$$\text{Sim}(e_i, e_j | z_i) = \text{Sim}_v(e_i, e_j) \cdot \text{Sim}_c(e_i, e_j, z_i). \quad (5)$$

Here, $\text{Sim}_v(e_i, e_j)$ is the visual similarity between the elements, computed based on their perceptual distance returned by DreamSim [47], a state-of-the-art model for visual similarity assessment. $\text{Sim}_c(e_i, e_j, z_i)$ is the contextual similarity between the elements. In our implementation, z_i is the subtree of the scene tree rooted at e_i 's parent. Following Lu *et al.* [4], we compute the Jaccard similarity between the elements in z_i and those in the subtree rooted at e_j 's parent. The Jaccard similarity is defined as the ratio of the intersection to the union of the elements in the two subtrees, where two elements are equal if they belong to the same category and have similar sizes. To improve efficiency, propagation is performed only for

elements whose similarity exceeds a default threshold of 0.2. We determined this value through a grid search over thresholds from 0.05 to 0.50 with a step size of 0.05. Each threshold was evaluated on the same set of 500 randomly sampled element pairs, including 250 pairs where propagation was appropriate and 250 pairs where propagation was not, and we selected the threshold with the highest F1 score.

To verify the impact of the propagation, users can click $\Rightarrow$ to overlay a flow map [48] on the grid visualization (Fig. 10). Each arrow in the flow map points to an infographic affected by the propagation, with its width indicating the number of propagated elements.

VI. EVALUATION

We first conducted quantitative experiments to evaluate the effectiveness of the scene-tree-based automatic selection and the treemap-based grid layout. We then demonstrated the usefulness of EleDetective through two use cases.

A. Quantitative Evaluation

1) Scene-Tree-Based Automatic Selection:

Datasets and model. We conducted experiments on three datasets. The first was the InfoDet test set [1], containing 5,000 infographics with bounding-box annotations for charts and human-recognizable objects (HROs), including icons and images. The second was ChartGalaxy [3], from which we sampled 4,377 synthetic infographics with bounding-box annotations for 11 chart categories, marks, 3 reference element categories (gridline, legend, axis), 3 textual element categories (title, source, annotation), and both data-relevant and irrelevant HROs. The third dataset was the Rico test set [49], employed to evaluate generalizability beyond infographics. It contains 13,108 user interfaces with bounding-box annotations for 25 component categories. For InfoDet and Rico, we used the state-of-the-art InternImage-based detection model fine-tuned by Zhu *et al.* [1]. For ChartGalaxy, we fine-tuned the same model under the identical setting.

Experimental setup. We compared our scene-tree-based automatic selection with four commonly used post-processing methods for detection: 1) Confidence Thresholding, which selects elements with confidence scores above a specified threshold; 2) NMS, which removes misselected elements by keeping only the one with the highest confidence score among overlapping elements; 3) Soft-NMS [50], which improves upon NMS by decreasing the confidence scores of overlapping elements rather than removing them; and 4) WBF [51], which merges overlapping elements by computing a confidence-weighted average of their positions. To ensure a fair comparison, we used a unified confidence threshold for all methods, chosen from the Precision-Recall curve to balance precision and recall. Other hyperparameters, such as the IoU threshold for judging overlap, were optimized via grid search.

Evaluation criteria. The performance was evaluated using two metrics: 1) mAP, a widely used metric calculated as the area under the precision-recall curve; and 2) LRP [52], which jointly considers localization accuracy and the number of false positives and negatives.

TABLE I

EVALUATION OF OUR SCENE-TREE-BASED AUTOMATIC SELECTION AND THE BASELINES. THE BEST ONE IS **BOLD**, AND THE RUNNER-UP IS UNDERLINED. IOU THRES. DENOTES THE OPTIMAL THRESHOLD FOR JUDGING OVERLAP DETERMINED VIA GRID SEARCH.

Method	mAP $\uparrow$	LRP $\downarrow$	Method	mAP $\uparrow$	LRP $\downarrow$	Method	mAP $\uparrow$	LRP $\downarrow$
Conf. Thres.	80.0	25.4	Conf. Thres.	71.1	37.4	Conf. Thres.	49.6	58.4
NMS (IoU thres.=0.5)	79.9	25.2	NMS (IoU thres.=0.85)	70.7	32.4	NMS (IoU thres.=0.95)	45.1	60.7
Soft-NMS (IoU thres.=0.5)	80.0	25.4	Soft-NMS (IoU thres.=0.85)	71.3	36.4	Soft-NMS (IoU thres.=0.95)	49.6	58.2
WBF (IoU thres.=0.95)	80.0	25.4	WBF (IoU thres.=0.95)	70.4	36.4	WBF (IoU thres.=0.95)	47.8	58.0
Ours	83.4	24.4	Ours	74.1	31.6	Ours	49.7	57.8

(a) InfoDet (b) ChartGalaxy (c) Rico

Results and analysis. Table I compares the performance of our method with the baselines. On the infographic datasets, InfoDet and ChartGalaxy, our method performed better than the baselines. For example, it achieved an mAP of 83.4% and an LRP of 24.4% on InfoDet, exceeding the best baseline performance of 80.0% mAP and 25.2% LRP. On the Rico dataset, although our method is not specifically designed for user interfaces, it still achieved better performance than all baselines, demonstrating its generalizability. Qualitative analysis showed that, unlike the baselines that primarily remove or merge misselected elements, our method corrects both misselected and missed elements. The detailed analysis is provided in Sec. I of the supplemental material.

2) Treemap-Based Grid Layout:

Datasets and models. We conducted experiments on two types of datasets. The first was infographic datasets, for which we used InfoDet [1]. We extracted the features of their scene trees using a Tree-LSTM model and clustered them accordingly. The second was natural image datasets employed by Zhou *et al.* [26], including CIFAR100 [53], iNat2021-mini [54], and ImageNet-1K [55]. We extracted features using pretrained models and clustered images based on their predicted classes. Please refer to Sec. II-A of the supplemental material for details on dataset statistics and pretrained models. **Experimental setup.** We compared our method with state-of-the-art grid and treemap layout methods. For the grid layout methods, we used the method proposed by Zhou *et al.* [26]. For the treemap layout methods, we used Nmap-AC/-EW [19], Weighted Maps [18], and SOT [17]. All methods were evaluated with grid sizes of 30×30, 40×40, and 50×50. For fair comparison, all methods received the same inputs: top-level clusters generated by the hierarchical exploration method, each represented by its center and number of samples.

Evaluation criteria. We used five measures:

- 1) **Proximity** of the samples, calculated as the area under the k -nearest neighbor preservation curve [20]. For each k , this curve shows the average number of preserved neighbors (*i.e.*, those appearing among the k nearest neighbors in both the high-dimensional space and the layout). Following Zhou *et al.* [26], we set $k = 50$.
- 2) **Compactness** of the clusters, calculated as the average squared distance between the grid cells of the samples and their centroids following Rottmann *et al.* [13].
- 3) **Convexity** of the clusters, calculated as the average ratio between the perimeter of each cluster's convex hull and the perimeter of the cluster itself [56].
- 4) **Displacement** of the cluster centers, calculated as the

TABLE II
EVALUATION OF THE LAYOUT METHODS.

Method	Prox. $\uparrow$	Comp. $\downarrow$	Conv. $\uparrow$	Disp. $\downarrow$	Area Pres. $\downarrow$
Zhou <i>et al.</i> 's	<u>412.8</u>	0.013	<u>0.956</u>	0.052	0.00
Nmap-AC	-	0.013	1.000	0.046	14.05
Nmap-EW	-	<u>0.012</u>	1.000	<u>0.047</u>	11.06
Weighted Maps	-	<u>0.012</u>	1.000	0.046	10.86
SOT	-	<u>0.012</u>	1.000	0.058	6.82
Ours	420.3	0.011	1.000	0.046	<u>5.86</u>

area-weighted average of the squared distances between input and final cluster centers [19].

- 5) **Area preservation** of the clusters, calculated by sorting both the input and final numbers of samples, and counting the number of cluster pairs whose relative order differs between the two sorted sequences.

Since the treemap layout methods partition the grid without assigning samples to cells, their proximity is not evaluated.

Results and analysis. Table II shows the performance of the layout methods, averaged across datasets and grid sizes. The full results for each grid size are available in Sec. II-B of the supplemental material. Compared to Zhou *et al.*'s method, our method achieves perfect convexity by mapping clusters to rectangular areas. While it slightly compromises area preservation, it improves proximity, compactness, and displacement. Compared to the baseline treemap layout methods, our method maintains comparable displacement and area preservation while achieving better compactness by incorporating it into the optimization.

Fig. 8 shows the generated layouts on CIFAR100 for a 30×30 grid. The full results across datasets are provided in Sec. II-C of the supplemental material. Zhou *et al.*'s method maps clusters to non-convex areas (Fig. 8(b)), which complicates the cluster boundaries and hinders analysis. The baseline treemap layout methods generate rectangular areas (Fig. 8(c)(d)), with SOT generating the most compact ones (Fig. 8(d)). However, SOT does not preserve the relative positions of the clusters, which hinders the interpretation of cluster relationships. In contrast, our method generates convex, compact areas that effectively preserve cluster positions (Fig. 8(e)), thereby enhancing cluster perception and facilitating their analysis.

B. Use Cases

We conducted two use cases to demonstrate the effectiveness of EleDetective in: 1) refining training data for

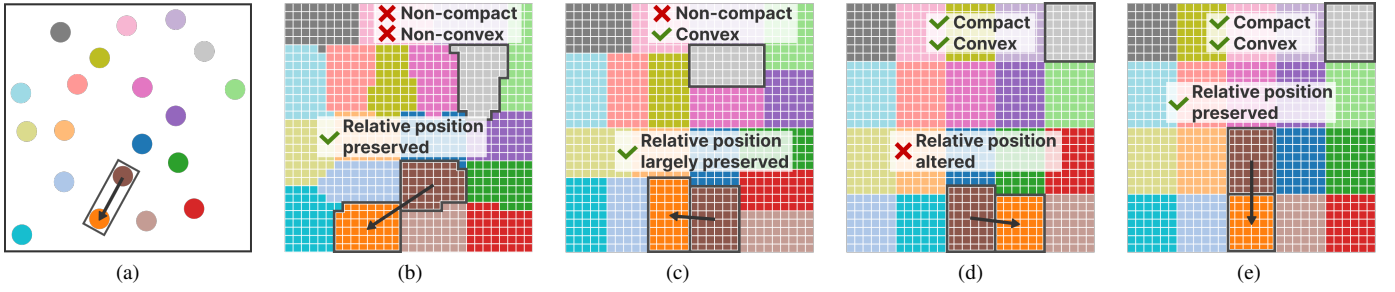


Fig. 8. Example of the input cluster centers and the grid visualizations generated by the evaluated methods: (a) the input cluster centers; (b) Zhou *et al.*'s method; (c) Weighted Maps; (d) SOT; (e) our treemap-based grid layout method.

infographic element detection; and 2) improving the visually grounded reasoning of VLMs.

1) Refining training data for Infographic Element Detection: In this use case, we collaborated with E1, who aimed to detect basic elements in real infographics from ChartGalaxy [3], including charts, marks, reference elements, textual elements, and HROs. To this end, he used the detection model described in Sec. VI-A1, which was trained on synthetic infographics from ChartGalaxy. E1 manually annotated 100 real infographics as a held-out evaluation subset, each with bounding-box annotations for these elements. The model achieved an mAP of 43.5% on this subset, which was below the expected 50.0%. Therefore, he decided to prepare a new dataset of 10,000 real infographics with bounding-box annotations to fine-tune the model.

Applying automatic element selection. Since manually annotating infographic elements requires domain expertise and is time-consuming, E1 applied the scene-tree-based target element selection in EleDetective. This automatic selection method increased the mAP to 48.1%. Unsatisfied with the performance, E1 used the visualizations in EleDetective to refine the annotations interactively.

Correcting missed axes and misselected HROs. E1 began by examining the grid visualization. He noticed an unusual selection pattern that suggested potential errors: no marks or reference elements were selected in the left region of a chart (Fig. 9A). By zooming into its cluster and examining infographics with similar selection results, he found that this region often contained texts and HROs, such as country names and flag icons, that indicate the category represented by each bar (Fig. 9A1). He identified this region as a missed categorical axis of the bar chart, which required correction. Additionally, he noticed that several license icons in the bottom-left corner were covered by a single large bounding box labeled as a data-irrelevant HRO (Fig. 9A2). This was a granularity error, as the icons should have been annotated separately rather than merged into one box. In the scene tree visualization, the larger markers indicated that these license icons had high propagation influence (Fig. 9A3), highlighting their prevalence and the need for correction. To correct these errors, E1 increased the confidence scores of the bounding boxes for the missed categorical axis and the individual license icons, and decreased the confidence score of the bounding box that incorrectly merged multiple license icons. He then used

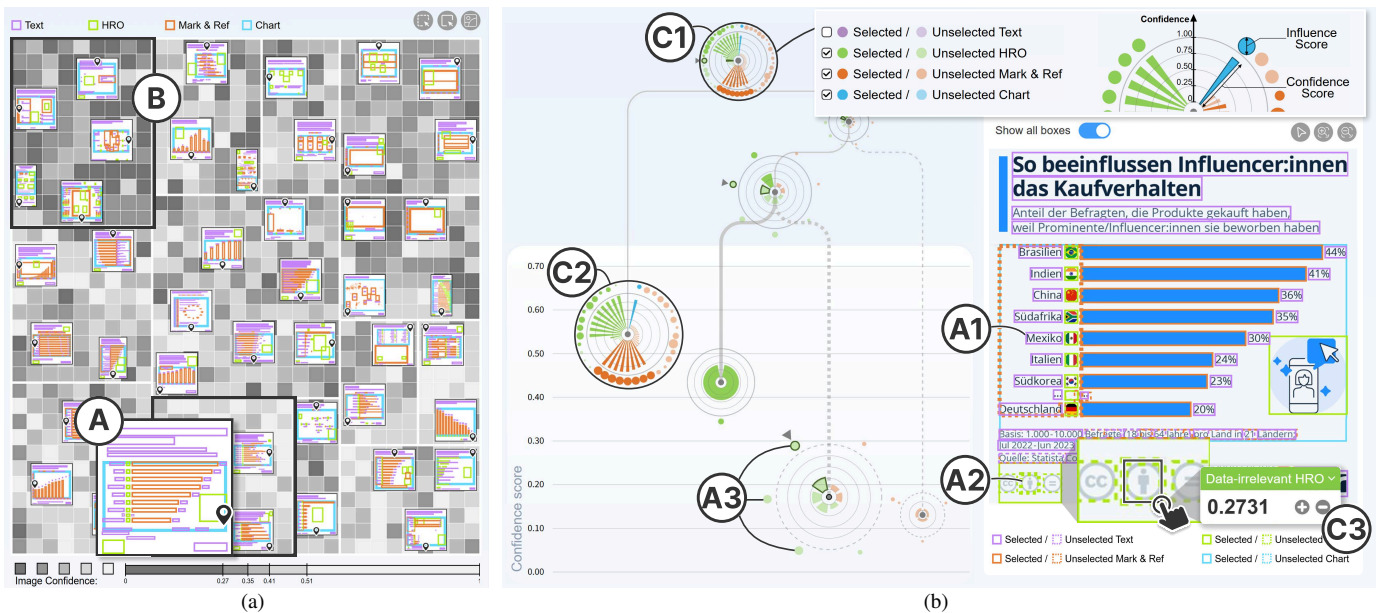


Fig. 9. EleDetective: (a) the grid visualization provides an overview by clustering infographics with similar selection results; (b) the scene tree visualization shows the selection result of a single infographic and summarizes key statistics of its regions. Both visualizations include legends to explain their visual encodings. The larger versions of both visualizations are provided in Sec. III of the supplemental material.

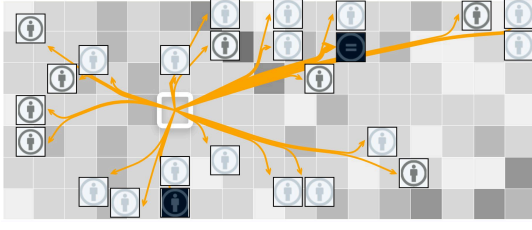


Fig. 10. The flow map for the correction of license icons.

the flow map to examine the propagation targets and adjust incorrect ones before applying the propagation (Fig. 10). By repeating this process, he manually corrected the selection of 8 axes and 16 data-irrelevant HROs across 20 infographics, and propagated these corrections. After fine-tuning the detection model on the refined annotations and performing the automatic selection method, the mAP increased from 27.4% to 38.7% for categorical axes and from 14.7% to 30.0% for license icons. At the category level, the mAP increased from 49.5% to 54.5% for axes and from 24.3% to 35.9% for data-irrelevant HROs, resulting in an overall improvement from 48.1% to 49.6%.

Correcting misclassified maps. E1 then zoomed out to the top level of the grid visualization to identify additional annotation errors. A cluster with darker cells (Fig. 9B) drew his attention, as the dark color indicated lower overall confidence and thus suggested potential errors. To identify the source of the lower confidence, he zoomed into the cluster and examined an infographic using the scene tree visualization. The scene tree showed that the low overall confidence mainly resulted from map fragments that were detected as candidates for marks with low confidence scores, as indicated by the short bars in the circular glyph (Fig. 11A). Most of the detected map fragments were filtered out, while the fragment covering the entire map was selected as a mark. E1 noted that prior research [3], [57] typically does not classify maps as marks or charts, and they should instead be labeled as data-relevant HROs. To address this issue, he manually corrected the category labels of 61 bounding boxes of map fragments across 57 infographics from

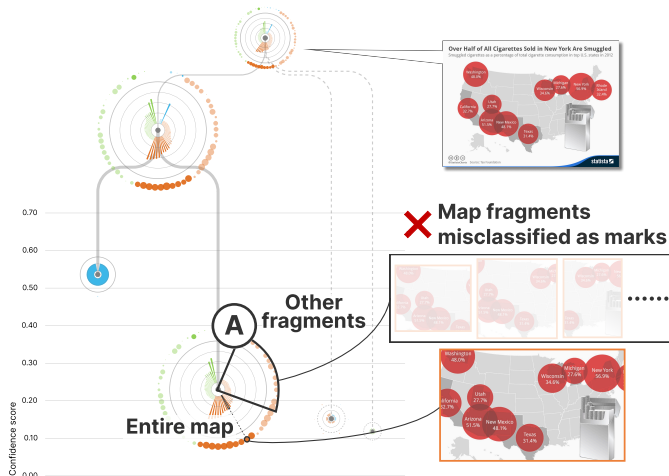


Fig. 11. Misclassified map fragments in the scene tree visualization.

TABLE III
COMPARISON OF ANNOTATION REFINEMENT EFFECTIVENESS BETWEEN ELEDETECTIVE AND THE ABLATED BASELINES.

Method	# Manual	# Propagated	Δ mAP
Baseline (Zhou <i>et al.</i> 's)	92	0	+1.4
Baseline (Weighted Maps)	83	0	+1.2
EleDetective	85	2,666	+2.9

marks to data-relevant HROs and propagated these corrections. After fine-tuning the detection model on the refined annotations and performing the automatic selection method, the mAP increased from 5.8% to 37.4% for maps. At the category level, the mAP increased from 80.4% to 82.5% for marks and from 73.4% to 76.8% for data-relevant HROs. This led to an overall improvement of mAP from 49.6% to 51.0%, exceeding the expected mAP of 50.0%.

In total, the overall mAP improved from 43.5% to 51.0% by correcting the selection of 85 elements across 77 infographics.

Post analysis. After the use case, we further evaluated the annotation refinement effectiveness of EleDetective and the transferability of the refined model.

To evaluate the annotation refinement effectiveness, we compared EleDetective with two ablated baselines that disabled correction propagation and simplified the visualization components. Specifically, both baselines removed the node-link diagram from the scene tree visualization and disabled correction propagation. The first baseline further replaced the treemap-based grid layout with Zhou *et al.*'s method [26], a representative grid layout method, while the second replaced it with Weighted Maps [18], a representative treemap layout method. Under the same 60-minute time constraint for interactive refinement, we evaluated EleDetective and the baselines by measuring the number of manual and propagated corrections, as well as the resulting mAP improvement. Table III shows the evaluation results, which indicate two benefits of EleDetective. First, even with the additional time required to verify propagated corrections, the number of manual corrections with EleDetective remained comparable to the baselines, indicating that the coordinated visualizations supported efficient manual correction. Second, these manual corrections were further propagated across infographics, resulting in more total corrections and a larger mAP improvement than the baselines. These results provide supporting evidence for the practical value of the coordinated visualizations and correction propagation within the EleDetective workflow.

To evaluate whether the refined model can be transferred to other domains, we compared the model before and after interactive refinement when fine-tuned on three out-of-domain datasets: Rico [49] with user interfaces, DocGenome [58] with document layouts, and PosterLayout [59] with poster designs. For each dataset, we fine-tuned each model on the training set for 12 epochs with a batch size of 16, using the AdamW optimizer [60] with a learning rate of 0.0001, and evaluated the mAP on the test set. As shown in Table IV, the refined model consistently improved mAP across all three datasets, indicating better transferability.

TABLE IV
PERFORMANCE OF THE DETECTION MODEL WHEN FINE-TUNED ON
OUT-OF-DOMAIN DATASETS.

Model	Rico	DocGenome	PosterLayout
Before refinement	39.4	71.4	55.5
After refinement	40.2	72.3	56.3
Δ mAP	+0.8	+0.9	+0.8

2) Improving the Visually Grounded Reasoning of VLMs:

The second use case was conducted with E2, who studies visually grounded reasoning for chart question answering. In this setting, a VLM reasons over bounding-box annotations of elements to produce the final answer (Fig. 12). He considered three types of elements: charts and HROs annotated with the model developed by Zhu *et al.* [1], and texts annotated with PP-OCRv4 [61]. To evaluate the visually grounded reasoning method, E2 tested it on the ChartQAPro benchmark [62], which contains charts and related questions with predefined ground-truth answers. He used GPT-4.1 as the VLM and obtained an accuracy of 58.8% by comparing its outputs with the ground-truth answers. Unsatisfied with the accuracy, E2 decided to analyze and refine the annotations using the visualizations in EleDetective. To better support chart question-answering, we made two adaptations: 1) Cell colors encoded accuracy, with darker cells representing lower accuracy; and 2) Besides clustering samples by selection results, we introduced a new clustering mode based on question-answer pairs.

Analyzing clusters formed by selection results. E2 first examined the grid visualization in the selection-result-based clustering mode. He noticed two large clusters with lower accuracies, mostly consisting of samples with only one bar chart (Fig. 13A) or line chart (Fig. 13B). The low accuracy of these common and visually simple chart types prompted E2 to examine the samples more closely. He found unevenly spaced bounding boxes for texts within the chart region (Fig. 13A1) due to missed detections of slanted texts. To address this, E2 ensembled PP-OCRv4 with AzureOCR [63], which can detect such texts more effectively. This improvement increased the accuracy from 58.8% to 59.6%.

Despite the improved OCR results, the accuracy for these clusters remained relatively low. Further analysis revealed that many questions in these simple charts required direct value

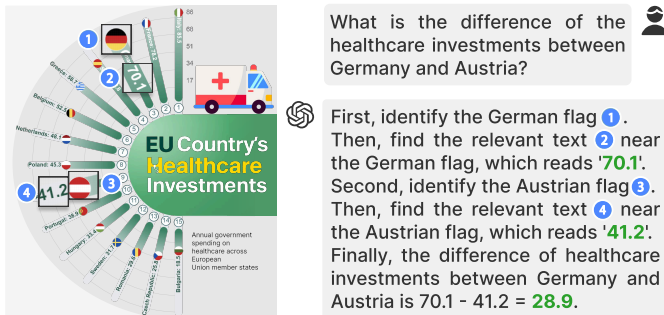


Fig. 12. The visually grounded reasoning of VLMs.

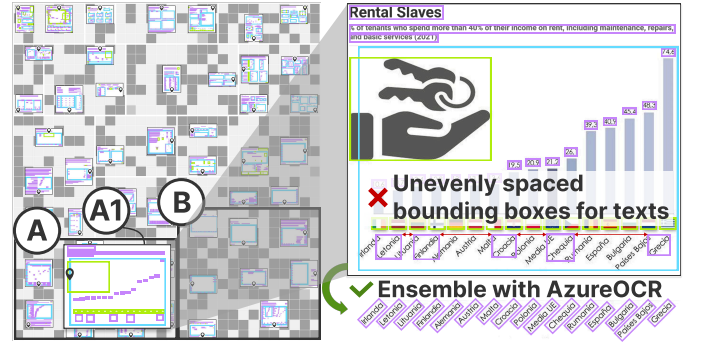


Fig. 13. Analyzing clusters formed by selection results and recovering missed texts in the OCR results.

extraction. This is a common yet challenging task in VLM-based chart question answering [2], [62]. E2 identified two failure cases that contributed to the low accuracy: 1) For charts with data value annotations, the bounding boxes occluded the annotations in low-resolution images, preventing the VLM from recognizing them (Fig. 14(a)); and 2) For charts without data value annotations, the VLM struggled to infer values by aligning marks with the axes (Fig. 14(b)). To mitigate these issues, E2 applied targeted improvements: 1) Adjusting the box-line widths with image resolution, which reduced occlusion in low-resolution images; 2) Adding auxiliary lines to charts without data value annotations to help the VLM align marks with the axes. The accuracy increased from 59.6% to 60.4% after adjusting line widths, and further to 60.6% after adding auxiliary lines.

Analyzing clusters formed by question-answer pairs. To identify other causes of low accuracy, E2 switched to the question-answer-based clustering mode. In this mode, he noticed two large clusters with lower accuracy (Fig. 15A, B) and examined them separately.

For cluster A, many questions required list-based answers, such as “[content marketing, email marketing, social media].” Generating a complete list is inherently more challenging than answering questions that require only a single response,

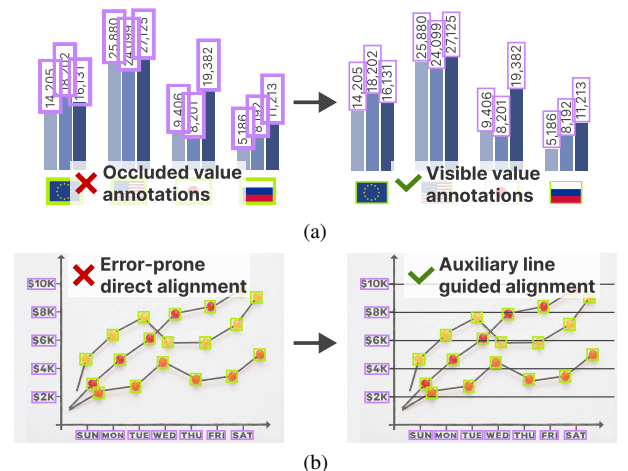


Fig. 14. Mitigating issues in value extraction: (a) scaling the line widths of detection boxes; (b) adding auxiliary lines for axis alignment.

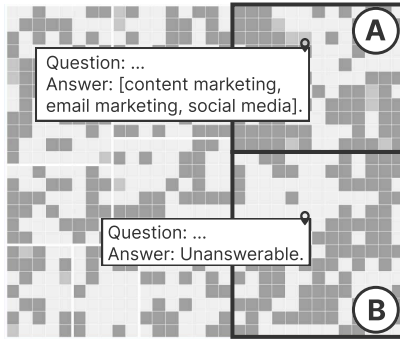


Fig. 15. Analyzing clusters formed by question-answer pairs.

such as multiple-choice questions. Therefore, E2 considered the low accuracy of cluster A reasonable. For cluster B, most questions were labeled as “unanswerable,” indicating that they referred to information absent from the chart. For example, one question referred to “Jun–Aug 2014,” but the chart only contained data for “Jun–Aug 2013.” To address this, E2 revised the instruction provided to the VLM, explicitly asking it to “read the question more carefully and ensure that all referenced information is present in the chart.” This revision increased the accuracy from 60.6% to 61.8%.

Overall, EleDetective helped E2 identify failure patterns in visually grounded reasoning and apply targeted improvements to text detection, value extraction, and instruction design, increasing the accuracy from 58.8% to 61.8%.

VII. EXPERT FEEDBACK AND DISCUSSION

Following the use cases, we conducted semi-structured interviews with the three experts from the requirement analysis (E1-E3) and four newly invited ones (E4-E7). The newly invited experts are two Ph.D. students (E4 and E5) and two professors (E6 and E7) specializing in visual analytics and machine learning. For those not involved in the use cases, we briefly introduced our tool and the use cases before the interview. Each interview lasted 45-70 minutes. More details on the interview protocol are provided in Sec. IV of the supplemental material. The experts were generally positive about the usability of EleDetective. They also identified limitations that suggest directions for future research.

A. Usability

Intuitive visual design and interaction. All experts agreed that the visual design and interaction of EleDetective are understandable after a brief introduction. E1 appreciated mapping clusters to rectangular areas in the grid visualization, noting: “The grid provides a clear overview of the selection results, and showing representative samples makes it easy to spot problematic clusters.” E3 found the scene tree visualization effective for “organizing elements, making it easier to correct errors.” E6 considered it intuitive to correct selection results by adjusting confidence scores in the scene tree visualization. E1, E2, and E6 all recognized that the coordinated visualizations and interactive refinement established a clear workflow for iteratively refining the selection results. Although we did not

conduct a dedicated user study to evaluate the visual design and interaction, such as the readability of the circular glyph, the expert feedback provides qualitative evidence regarding their usability in the intended workflow. A dedicated evaluation of glyph readability remains future work.

Reducing correction effort. E4 was impressed by the effectiveness of correction propagation, especially when correcting the selection of license icons in the first use case: “It’s surprising that adjusting the confidence scores of only 16 icons could raise the mAP from 24.3% to 35.9% for data-irrelevant HROs.” E1 found the flow map helpful: “The flow map clearly shows how corrections propagate, helping me confirm whether the propagation is correct.”

Supporting infographic generation. Although we focused on refining training data for infographic element detection and infographic understanding in the use cases, E5 suggested that EleDetective could readily support template-based infographic generation. Specifically, the grid visualization facilitates template construction by revealing recurring layout patterns in real infographics, which can be summarized into reusable layout templates for placing new elements when generating infographics. E7 further noted that the constructed scene tree can support example-based infographic generation by recording how elements are grouped together, which can guide layout adjustments when newly generated elements differ in size from those in the original example.

B. Limitations and Future Work

Propagation scalability. Propagating a correction requires evaluating the similarity between the corrected element and all other elements. For the datasets considered in the use cases, which contain fewer than 10,000 infographics, this process can be completed in real-time. However, when scaling to millions of infographics, propagation may take several minutes. Therefore, acceleration of the propagation process warrants further exploration. One potential solution is to perform feature-based filtering before propagation, limiting similarity evaluation to only the nearest neighbors of the corrected element in the feature space.

Addressing localization errors. Our tool addresses common errors in target element selection, including missed, misselected, and misclassified elements. E6 mentioned that users may also need to correct localization errors, where predicted bounding boxes do not accurately align with object boundaries. Extending our tool to support the correction of localization errors presents challenges. First, identifying potential localization errors requires modeling the uncertainty of each box boundary. Second, propagating corrections to box boundaries requires a dedicated model capable of capturing and transferring these subtle adjustments.

Supporting new category creation. Our study focuses on a common object detection setting, where users define the target element categories of interest in advance and refine annotations for these categories. In broader annotation settings, users may also encounter unknown objects or new semantic categories during correction. A potential solution for such cases is to develop a dedicated view that highlights elements

with low confidence scores and enables users to label them with newly created categories. Such a solution would also require propagating the newly created category labels to similar elements and incorporating a lightweight classifier that adapts the detector to recognize the new categories [38].

VIII. CONCLUSION

In this paper, we introduce EleDetective, a visual analysis tool for interactive target element selection in infographics. EleDetective introduces a scene-tree-based method for automatic target element selection. This method is complemented by a coordinated visualization design that combines a treemap-based grid layout and a scene tree visualization to support the identification and examination of the selection errors. Interactive refinement further allows users to correct identified errors, with their corrections propagated across infographics using query-sensitive similarity. Quantitative evaluation demonstrates the effectiveness of the scene-tree-based automatic selection and the treemap-based grid layout. Two use cases further showcase EleDetective's utility in refining training data for infographic element detection and improving visually grounded reasoning of VLMs.

ACKNOWLEDGMENTS

The authors would like to thank Duan Li, Zhen Li, and Lanxi Xiao for their valuable discussions and comments.

REFERENCES

- [1] J. Zhu, Y. Zhou, Z. Wang, J. Yao, Y. Gu, Y. Yuan, and S. Liu, "InfoDet: A dataset for infographic element detection," in *Proceedings of the International Conference on Learning Representations*, 2026.
- [2] X. Li, Y. Sun, W. Cheng, Y. Zhu, and H. Chen, "Chain-of-region: Visual language models need details for diagram analysis," in *Proceedings of the International Conference on Learning Representations*, 2025.
- [3] Z. Li, D. Li, Y. Guo, X. Guo, B. Li, L. Xiao, S. Qiao, J. Chen, Z. Wu, H. Zhang, X. Shu, and S. Liu, "ChartGalaxy: A dataset for infographic chart understanding and generation," in *Proceedings of the International Conference on Learning Representations*, 2026.
- [4] M. Lu, C. Wang, J. Lanir, N. Zhao, H. Pfister, D. Cohen-Or, and H. Huang, "Exploring visual information flows in infographics," in *Proceedings of the CHI conference on human factors in computing systems*, 2020, pp. 1–12.
- [5] D. P. Papadopoulos, J. R. Uijlings, F. Keller, and V. Ferrari, "We don't need no bounding-boxes: Training object class detectors using only human verification," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 854–863.
- [6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proceedings of European Conference on Computer Vision*, 2014, pp. 740–755.
- [7] M. Li, A. G. Patil, K. Xu, S. Chaudhuri, O. Khan, A. Shamir, C. Tu, B. Chen, D. Cohen-Or, and H. Zhang, "Grains: Generative recursive autoencoders for indoor scenes," *ACM Transactions on Graphics*, vol. 38, no. 2, pp. 1–16, 2019.
- [8] Z.-H. Zhou and H.-B. Dai, "Query-sensitive similarity measure for content-based image retrieval," in *Proceedings of International Conference on Data Mining*, 2006, pp. 1211–1215.
- [9] C. Chen, J. Yuan, Y. Lu, Y. Liu, H. Su, S. Yuan, and S. Liu, "OoD-Analyzer: Interactive analysis of out-of-distribution samples," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 7, pp. 3335–3349, 2021.
- [10] J. Wang, S. Liu, and W. Zhang, "Visual analytics for machine learning: A data perspective survey," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 12, pp. 7637–7656, 2024.
- [11] M. Liu, J. Shi, Z. Li, C. Li, J. Zhu, and S. Liu, "Towards better analysis of deep convolutional neural networks," *IEEE Transactions on Visualization and Computer Graphics*, vol. 23, no. 1, pp. 91–100, 2017.
- [12] N. Quadrianto, A. J. Smola, L. Song, and T. Tuytelaars, "Kernelized sorting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 32, no. 10, pp. 1809–1821, 2010.
- [13] P. Rottmann, M. Wallinger, A. Bonerath, S. Gedicke, M. Nöllenburg, and J.-H. Haurert, "Mosaicsets: Embedding set systems into grid graphs," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 1, pp. 875–885, 2023.
- [14] G. Strong and M. Gong, "Self-sorting map: An efficient algorithm for presenting multimedia data in structured layouts," *IEEE Transactions on Multimedia*, vol. 16, no. 4, pp. 1045–1058, 2014.
- [15] O. Fried, S. DiVerdi, M. Halber, E. Sizikova, and A. Finkelstein, "Iso-match: Creating informative grid layouts," *Computer Graphics Forum*, vol. 34, no. 2, pp. 155–166, 2015.
- [16] X. Liu, Y. Hu, S. North, and H.-W. Shen, "CorrelatedMultiples: Spatially coherent small multiples with constrained multi-dimensional scaling," *Computer Graphics Forum*, vol. 37, no. 1, pp. 7–18, 2018.
- [17] J. Wood and J. Dykes, "Spatially ordered treemaps," *IEEE transactions on visualization and computer graphics*, vol. 14, no. 6, pp. 1348–1355, 2008.
- [18] M. Ghoniem, M. Cornil, B. Broeksema, M. Stefan, and B. Otjacques, "Weighted maps: treemap visualization of geolocated quantitative data," in *Proceedings of SPIE - Visualization and Data Analysis*, vol. 9397, 2015, pp. 163–177.
- [19] F. S. Duarte, F. Sikansi, F. M. Fatore, S. G. Fadel, and F. V. Paulovich, "Nmap: A novel neighborhood preservation space-filling algorithm," *IEEE transactions on visualization and computer graphics*, vol. 20, no. 12, pp. 2063–2071, 2014.
- [20] D. Bertucci, M. M. Hamid, Y. Anand, A. Ruangrotsakun, D. Tabatabai, M. Perez, and M. Kahng, "DendroMap: Visual exploration of large-scale image datasets for machine learning with treemaps," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 1, pp. 320–330, 2023.
- [21] D. Li, J. Yuan, X. Guo, X. Wang, Y. Liu, W. Yang, and S. Liu, "Ncp: Neighborhood-preserving non-uniform circle packing for visualization," 2026. [Online]. Available: <https://arxiv.org/abs/2602.00668>
- [22] C. Chen, Z. Wang, J. Wu, X. Wang, L.-Z. Guo, Y.-F. Li, and S. Liu, "Interactive graph construction for graph-based semi-supervised learning," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 9, pp. 3701–3716, 2021.
- [23] A. Hahnaut, R. Giot, R. Bourqui, and D. Auber, "Vrgrid: Efficient transformation of 2d data into pixel grid layout," in *Proceedings of the International Conference Information Visualisation*, 2022, pp. 11–20.
- [24] J. B. Tenenbaum, V. de Silva, and J. C. Langford, "A global geometric framework for nonlinear dimensionality reduction," *Science*, vol. 290, no. 5500, pp. 2319–2323, 2000.
- [25] Y. Zhou, W. Yang, J. Chen, C. Chen, Z. Shen, X. Luo, L. Yu, and S. Liu, "Cluster-aware grid layout," *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 1, pp. 240–250, 2024.
- [26] Y. Zhou, C. Chen, Z. Shen, J. Zhu, J. Chen, W. Yang, and S. Liu, "Hierarchical fuzzy-cluster-aware grid layout for large-scale data," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 10, pp. 8200–8213, 2025.
- [27] M. Bruls, K. Huizing, and J. J. van Wijk, "Squarified treemaps," in *Proceedings of the Joint EUROGRAPHICS and IEEE TCVG Symposium on Visualization*, 2000, pp. 33–42.
- [28] L. S. Snyder, Y.-S. Lin, M. Karimzadeh, D. Goldwasser, and D. S. Ebert, "Interactive learning for identifying relevant tweets to support real-time situational awareness," *IEEE Transactions on Visualization and Computer Graphics*, vol. 26, no. 1, pp. 558–568, 2020.
- [29] H. S. Garcia Caballero, M. A. Westenberg, B. Gebre, and J. J. van Wijk, "V-awake: a visual analytics approach for correcting sleep predictions from deep learning models," *Computer Graphics Forum*, vol. 38, no. 3, pp. 1–12, 2019.
- [30] M. Khayat, M. Karimzadeh, J. Zhao, and D. S. Ebert, "VASSL: A visual analytics toolkit for social spambot labeling," *IEEE Transactions on Visualization and Computer Graphics*, vol. 26, no. 1, pp. 874–883, 2020.
- [31] W. Yang, X. Ye, X. Zhang, L. Xiao, J. Xia, Z. Wang, J. Zhu, H. Pfister, and S. Liu, "Diagnosing ensemble few-shot classifiers," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 9, pp. 3292–3306, 2022.
- [32] L. Gou, L. Zou, N. Li, M. Hofmann, A. K. Shekar, A. Wendt, and L. Ren, "Vatld: A visual analytics system to assess, understand and improve traffic light detection," *IEEE transactions on visualization and computer graphics*, vol. 27, no. 2, pp. 261–271, 2021.
- [33] J. Wang, Y. Li, Z. Zhou, C. Wang, Y. Hou, L. Zhang, X. Xue, M. Kamp, X. L. Zhang, and S. Chen, "When, where and how does it fail? a spatial-temporal visual analytics approach for interpretable object detection in

- autonomous driving,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 12, pp. 5033–5049, 2023.
- [34] C. Chen, Y. Guo, F. Tian, S. Liu, W. Yang, Z. Wang, J. Wu, H. Su, H. Pfister, and S. Liu, “A unified interactive model evaluation for classification, object detection, and instance segmentation in computer vision,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 1, pp. 76–86, 2024.
- [35] W. He, L. Zou, A. K. Shekar, L. Gou, and L. Ren, “Where can we help? A visual analytics approach to diagnosing and improving semantic segmentation of movable objects,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 1, pp. 1040–1050, 2022.
- [36] X. Xuan, X. Wang, W. He, J. P. Ono, L. Gou, K.-L. Ma, and L. Ren, “VISTA: A visual analytics framework to enhance foundation model-generated data labels,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 10, pp. 6991–7003, 2025.
- [37] C. Chen, J. Wu, X. Wang, S. Xiang, S.-H. Zhang, Q. Tang, and S. Liu, “Towards better caption supervision for object detection,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 4, pp. 1941–1954, 2022.
- [38] S. Jamonnak, J. Guo, W. He, L. Gou, and L. Ren, “OW-Adapter: Human-assisted open-world object detection with a few examples,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 30, no. 1, pp. 694–704, 2024.
- [39] Z.-T. Chen, Y. Wang, Q. Wang, Y. Wang, and H. Qu, “Towards automated infographic design: Deep learning-based auto-extraction of extensible timeline,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 26, no. 1, pp. 917–926, 2020.
- [40] Z. Tang, J. Chen, Z. Zhang, M. Zhou, Y. Liu, Y. Pu, Y. Bai, E. Smith, and Y. Yuan, “Masked region transformer for layered image generation and editing at scale,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026.
- [41] J. Koch and A. Oulasvirta, “Computational layout perception using gestalt laws,” in *Proceedings of the CHI Conference Extended Abstracts on Human Factors in Computing Systems*, 2016, pp. 1423–1429.
- [42] K. S. Tai, R. Socher, and C. D. Manning, “Improved semantic representations from tree-structured long short-term memory networks,” in *Proceedings of the Annual Meeting of the Association for Computational Linguistics and the International Joint Conference on Natural Language Processing*, 2015, pp. 1556–1566.
- [43] T. Kamada, S. Kawai *et al.*, “An algorithm for drawing general undirected graphs,” *Information processing letters*, vol. 31, no. 1, pp. 7–15, 1989.
- [44] T. M. Fruchterman and E. M. Reingold, “Graph drawing by force-directed placement,” *Software: Practice and experience*, vol. 21, no. 11, pp. 1129–1164, 1991.
- [45] W. Scheibel, D. Limberger, and J. Döllner, “Survey of treemap layout algorithms,” in *Proceedings of the international symposium on visual information communication and interaction*, 2020, pp. 1–9.
- [46] J. T. Vogelstein, J. M. Conroy, V. Lyzinski, L. J. Podrazik, S. G. Kratzer, E. T. Harley, D. E. Fishkind, R. J. Vogelstein, and C. E. Priebe, “Fast approximate quadratic programming for graph matching,” *PLOS one*, vol. 10, no. 4, p. e0121002, 2015.
- [47] S. Fu, N. Tamir, S. Sundaram, L. Chai, R. Zhang, T. Dekel, and P. Isola, “Dreamsim: Learning new dimensions of human visual similarity using synthetic data,” in *Proceedings of Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 50 742–50 768.
- [48] D. Phan, L. Xiao, R. Yeh, P. Hanrahan, and T. Winograd, “Flow map layout,” in *Proceedings of IEEE Symposium on Information Visualization*, 2005, pp. 219–224.
- [49] B. Deka, Z. Huang, C. Franzen, J. Hibschan, D. Afegan, Y. Li, J. Nichols, and R. Kumar, “Rico: A mobile app dataset for building data-driven design applications,” in *Proceedings of the annual ACM symposium on user interface software and technology*, 2017, pp. 845–854.
- [50] N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, “Soft-nms—improving object detection with one line of code,” in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 5561–5569.
- [51] R. Solovyev, W. Wang, and T. Gabruseva, “Weighted boxes fusion: Ensembling boxes from different object detection models,” *Image and Vision Computing*, vol. 107, p. 104117, 2021.
- [52] K. Oksuz, B. C. Cam, E. Akbas, and S. Kalkan, “Localization recall precision (lrp): A new performance metric for object detection,” in *Proceedings of the European conference on computer vision*, 2018, pp. 504–519.
- [53] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep. TR-2009, 2009.
- [54] G. Van Horn, E. Cole, S. Beery, K. Wilber, S. Belongie, and O. Mac Aodha, “Benchmarking representation learning for natural world image collections,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 12 884–12 893.
- [55] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.
- [56] M. Oppermann and T. Munzner, “Vizsnippets: Compressing visualization bundles into representative previews for browsing visualization collections,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 1, pp. 747–757, 2022.
- [57] C. Yang, C. Shi, Y. Liu, B. Shui, J. Wang, M. Jing, L. Xu, X. Zhu, S. Li, Y. Zhang, G. Liu, X. Nie, D. Cai, and Y. Yang, “Chartmimic: Evaluating lmm’s cross-modal reasoning capability via chart-to-code generation,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.09961>
- [58] R. Xia, S. Mao, X. Yan, H. Zhou, B. Zhang, H. Peng, J. Pi, D. Fu, W. Wu, H. Ye, S. Feng, B. Wang, C. Xu, C. He, P. Cai, M. Dou, B. Shi, S. Zhou, Y. Wang, B. Wang, J. Yan, F. Wu, and Y. Qiao, “Docgenome: An open large-scale scientific document benchmark for training and testing multi-modal large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.11633>
- [59] H. Y. Hsu, X. He, Y. Peng, H. Kong, and Q. Zhang, “Posterlayout: A new benchmark and approach for content-aware visual-textual presentation layout,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 6018–6026.
- [60] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proceedings of International Conference on Learning Representations*, 2019.
- [61] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, and H. Wang, “PP-OCR: A practical ultra lightweight OCR system,” 2020. [Online]. Available: <https://arxiv.org/abs/2009.09941>
- [62] A. Masry, M. S. Islam, M. Ahmed, A. Bajaj, F. Kabir, A. Kartha, M. T. R. Laskar, M. Rahman, S. Rahman, M. Shahmohammadi, M. Thakkar, M. R. Parvez, E. Hoque, and S. Joty, “ChartQAPro: A more diverse and challenging benchmark for chart question answering,” in *Findings of the Association for Computational Linguistics*, 2025, pp. 19 123–19 151.
- [63] Microsoft, “OCR - Optical Character Recognition,” <https://learn.microsoft.com/en-us/azure/ai-services/computer-vision/overview-ocr>, 2024, accessed: 2025-07-09.



Jiangning Zhu is a third-year Ph.D. student at the School of Software, Tsinghua University. His research interest is explainable artificial intelligence. He received a B.S. degree from Tsinghua University.



Yuxing Zhou is a third-year master's student at the School of Software, Tsinghua University. His research interest is layout algorithms. He received a B.S. degree from Tsinghua University.



Lei Li is a Ph.D. student at the School of Software, Tsinghua University. His research interest is explainable artificial intelligence. He received a B.S. degree from Wuhan University of Technology and an M.S. degree from Beihang University.



Shixia Liu is a professor at Tsinghua University. Her research interests include explainable artificial intelligence and visual analytics for big data. She worked as a research staff member at IBM China Research Lab and as a lead researcher at Microsoft Research Asia. She received a B.S. and M.S. from Harbin Institute of Technology, and a Ph.D. from Tsinghua University. She is a fellow of IEEE and an associate editor-in-chief of IEEE Trans. Vis. Comput. Graph.